

Virtual Memory

Module 6

Operating Systems

Chapter Outline

- Virtual memory
 - Mechanism of virtual memory
 - Address translation
 - Page fault
- Page replacement
- Page replacement algorithm
 - FIFO
 - Optimal LRU
 - Random page replacement
 - Least frequently used
 - Most frequently used
 - Non-used recently page replacement algorithm
- Paging Mechanism
- Additional techniques

Virtual Memory

- Why we need virtual memory
 - The program size may exceed the amount of physical memory
 - The OS keeps those parts of the program currently in use in main memory and the rest on disk
 - The piece of the program being swapped between disk and memory as needed
 - Virtual memory can also work in a multiprogramming system

Virtual Memory

- A technique that allows execution of a process without having to allocate it the memory space it needs, completely
- It gives the idea that large memory is available at programmer's disposal even computer has relatively small main memory
- A program can be written assuming large address space is available
- That is the virtual address space >> physical address space
- The entire logical address space of a process must be in physical memory before the process can execute
- First Implemented in Intel 80386

Virtual Memory

- Advantages of virtual memory:
 - Programs can be larger than the size of the physical memory
 - Virtual memory accommodates them
 - Virtual memory provides a logical view of memory which is larger than its physical size
- Virtual memory is not easy to implement
- May substantially decrease performance if it is used carelessly
- Virtual memory can be implemented via:
 - Paging
 - Segmentation

Virtual Memory

- Demand Paging

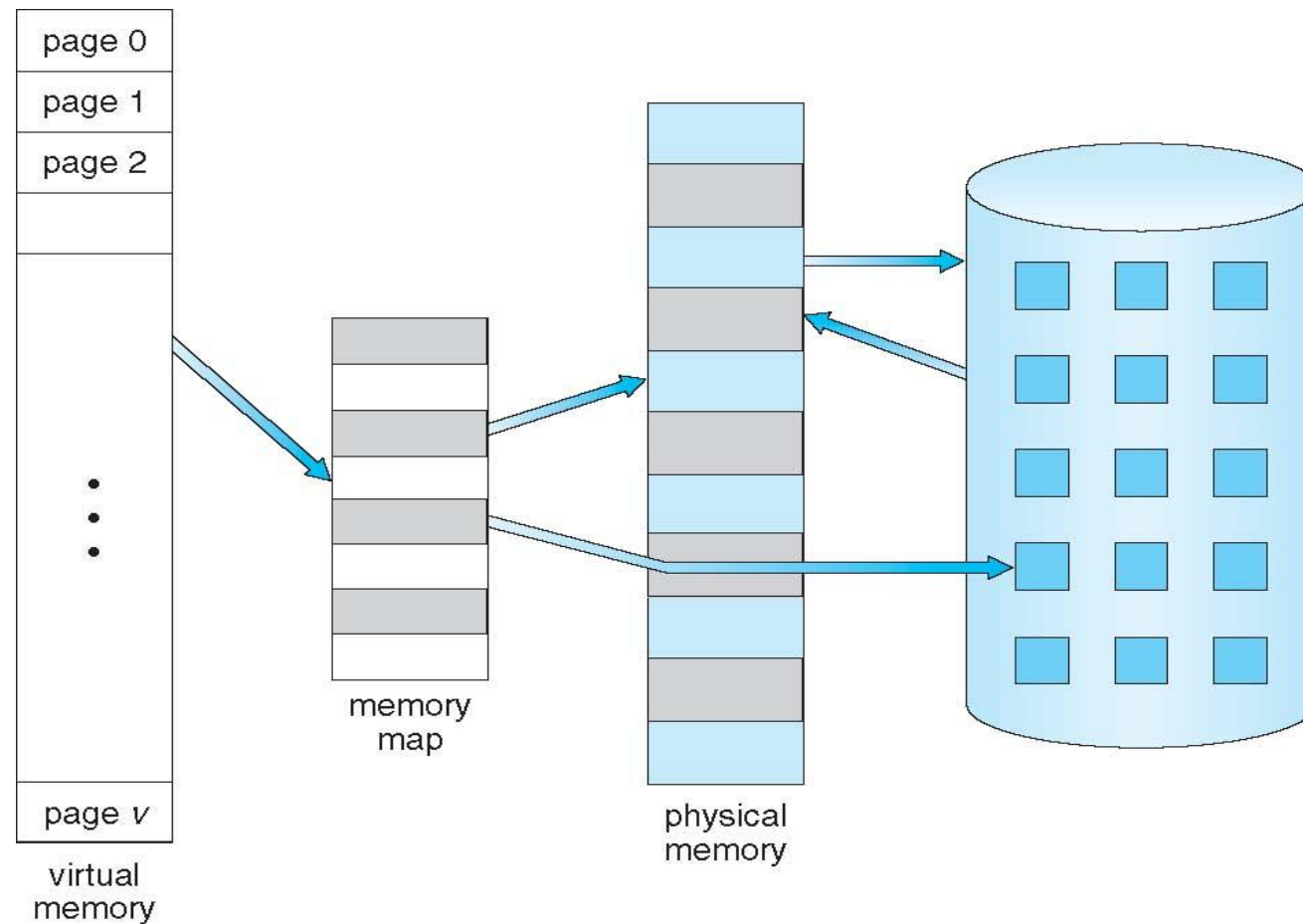
- In demand paging, bring a page into main memory only when it is needed
 - Less I/O needed
 - Less memory needed
 - Faster response
 - More users
- Page is needed → reference to it
 - Invalid reference → abort
 - Not in memory → bring to memory
- It can also be implemented in a segmentation system

- Demand Segmentation

- Demand segmentation can also be used to provide virtual memory
- Several systems provide paged segmentation scheme
 - Segments are broken into pages
- The IBM OS/2 uses demand segmentation

Virtual Memory

Virtual Memory is Larger Than Physical Memory



Mechanism of Virtual Memory

- When a new process is initiated, the system loader must load one page (start point) into real memory
- Execution of the process will commence
- This will continue as long as memory references generated by this page also within the same page
- If the virtual address generated will reference a page which is not in real memory; this is termed as page fault, which generates an interrupt
- The demand paging will be used, and system loader will load the next demanded page
- Execution will proceed after requested page is loaded
- By a series of page faults occurring in this way, a sub-set (resident set) of the pages of a process will gradually accumulate in real memory
- When the process terminates the OS releases all pages belonging to that process, making room for other processes

Mechanism of Virtual Memory

- Additional techniques
 - Hardware techniques to improve performance of virtual paged system
 - Translation look aside buffer (TLB)
 - Associative mapping

Translation Look Aside Buffer

TLB (Translation Lookaside Buffer)

- TLB is a **small, fast cache** that stores **recent page table entries**.

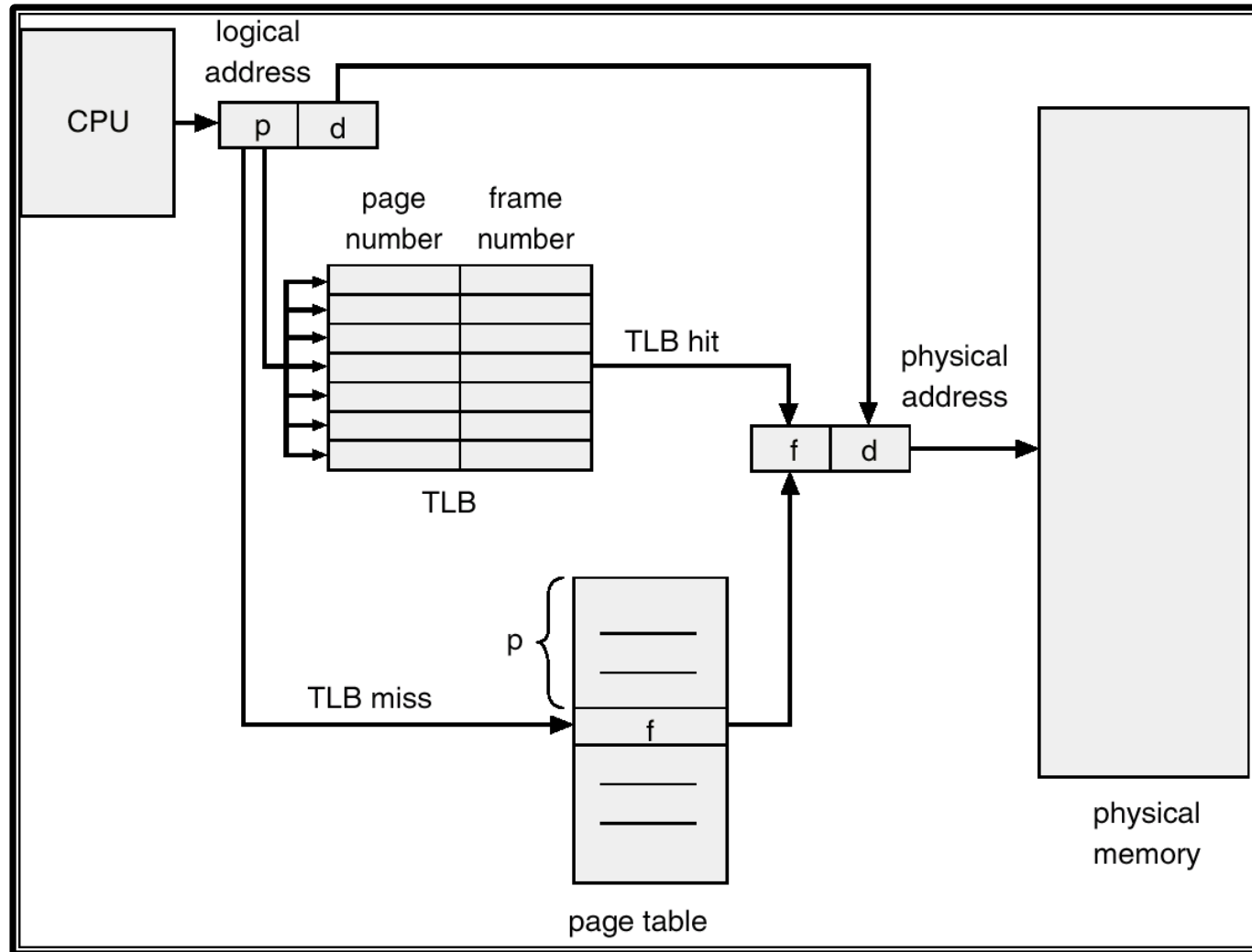
Key Points

- Stores **Virtual Page Number** → **Frame Number**
- Reduces **page table access time**
- Implemented using **associative memory**
- Part of the **MMU (Memory Management Unit)**

In Virtual Memory

- Speeds up **address translation**
 - Avoids accessing main memory page table
- **Associative Memory** is a **hardware concept** for fast searching
 - **TLB** is an **OS structure** that uses associative memory to speed up **address translation**

Translation Look Aside Buffer



Associative Mapping

Associative Memory is a **hardware memory type** that is accessed by **content rather than address**.

Key Points

- Searches **all entries in parallel**
- Returns a match based on **content**
- Very **fast**, but **expensive**
- Used where **quick lookup** is required

In Virtual Memory

- Associative memory is the **technology**
- TLB is **implemented using associative memory**
- **Example**
- If you search:

Virtual Page Number = 5

Associative memory compares **all stored page numbers simultaneously** and returns the matching frame.

Page Fault

- If the virtual address created a reference for a page which is not in real memory; it is termed as page fault
- Following measures should be done when a page fault occur
 - Get empty frame
 - Swap page into frame
 - Reset tables, present or valid bit = 1
 - Restart instruction

Page Replacement Algorithms

Page Replacement

- When a page fault occurs
 - The OS may have to remove a page from memory to make room for incoming page
 - If the page to be removed has been modified, it must be updated to disk
- For better performance, at each page fault
 - Pick a random page from memory that is not heavily used and replace it
 - If a heavily used page is removed, it may have to be brought back in quickly, resulting in extra overhead
- Use modify (dirty) *bit* to reduce overhead of page transfers – only modified pages are written to disk

Page Replacement

- Page replacement procedure
 - If no frame is free, we free a frame by writing its contents to disk
 - Change page table entry that the page is no longer in memory
 - Swap in new demanded page
 - Update the page table entry

Page Replacement Algorithms

- There are different page replacement algorithms
- Each OS has its own page replacement technique
- The important thing is, it should be one with lowest page fault rate
- Replacement policy/strategy is concerned with selecting a page currently in memory to be replaced
- All policies are based on, the page that is removed should be the page least likely to be referenced in the near future
- **Principal of locality** says, there is a high correlation between recent referencing history and near future referencing patterns
- Most policies try to predict future behavior on the basis of past behavior

FIFO Algorithm

- The FIFO maintain a list of all pages in memory with the page at the head the oldest one and page at the tail most recent arrival
- On a page fault, the page at the head is removed and the new page added to tail of the list
- Salient points
 - FIFO algorithm is very easy to implement
 - Its performance is not always good because it may replace pages heavily used throughout the life of a program
 - Low overhead paging algorithm

First-In-First-Out (FIFO) Algorithm

- Reference string: **7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1**
- 3 frames (3 pages can be in memory at a time per process)
- Can vary by reference string: consider 1,2,3,4,1,2,5,1,2,3,4,5
 - Adding more frames can cause more page faults!
- How to track ages of pages?
 - Just use a FIFO queue

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2																		
	0	0	0																		
		1	1																		

page frames

15 page faults

Optimal Algorithm

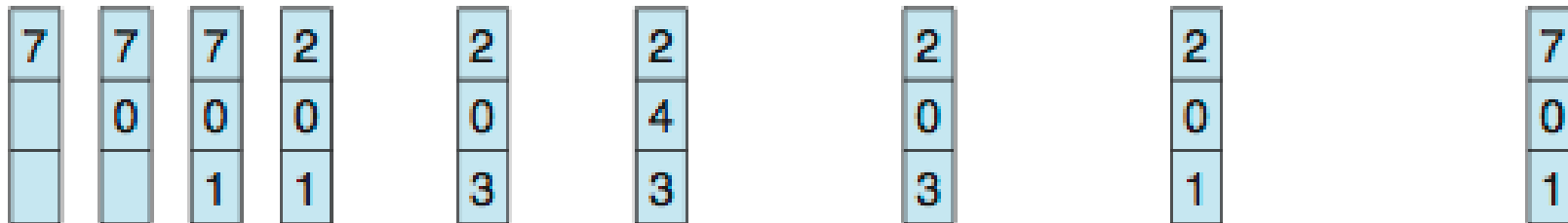
- Replace the page that will not be used for the longest period of time is called **principal of locality**
- This algorithm guarantees the lowest possible page fault rate
- Salient points
 - Optimal replacement is much better than FIFO
 - Difficult to implement, since it requires future knowledge of reference string
 - Not used in practical systems because it cannot implement
 - Optimal algorithm is mainly used for comparison studies

Optimal Algorithm

- Replace page that will not be used for longest period of time
 - 9 is optimal for the example
- How do you know this?
 - Can't read the future
- Used for measuring how well your algorithm performs

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1



page frames

9 page faults

Least Recently Used (LRU) Algorithm

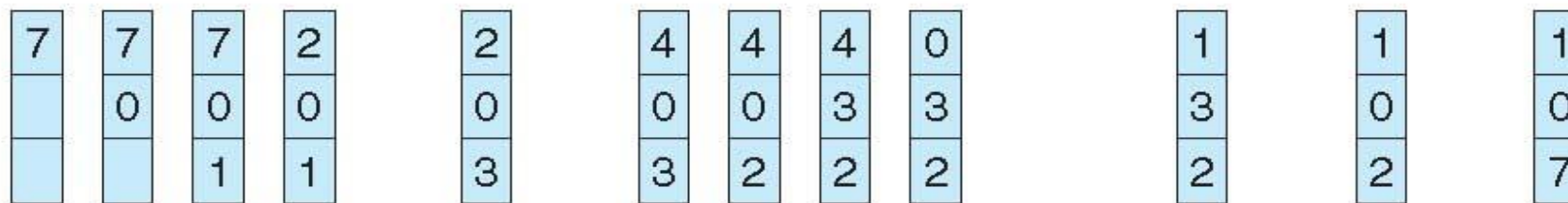
- Least recently used (LRU) algorithm chooses that page that has not been used for the longest period of time
- Salient points
 - LRU is quite good
 - Not easy to implement, the problem is to determine an order for the frames defined by the time of last use
 - Requires substantial hardware assistance because of tremendous overheads due to interrupts
- LRU feasible as a stack
 - Keep a stack of page numbers
 - Whenever a page is referenced, it is removed from the stack and put on top
 - In this way the top of the stack contains MRU page and bottom LRU page

Least Recently Used (LRU) Algorithm

- Use past knowledge rather than future
- Replace page that has not been used in the most amount of time
- Associate time of last use with each page
- 12 faults – better than FIFO but worse than OPT
- Generally good algorithm and frequently used
- But how to implement?

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1



page frames

12 page faults

Least Frequently Used (LFU) Algorithm

- LFU page replacement algorithm replaces the page from memory which is least frequently used
- It keeps a counter for the number of references to each page and the page with the smallest count is replaced
- This algorithm suffers from the situation in which a page initially used heavily, but then never again used

Least Recently Used (LRU) Algorithm – Example

Given

- Number of Page Frames = 3

Reference String: 7 0 1 2 0 3 0 4 2 3 0 3 2

Reference: 7 0 1 2 0 3 0 4 2 3 0 3 2

Frame 1: 7 7 7 2 2 2 2 4 4 4 0 0 0

Frame 2: - 0 0 0 0 0 0 0 0 3 3 3 3

Frame 3: - - 1 1 1 3 3 3 2 2 2 2 2

Page Faults: ✓ ✓ ✓ ✓ ✗ ✓ ✗ ✓ ✓ ✓ ✓ ✗ ✗

Total Page Faults = 9 Page Faults

Most Frequently Used (MFU) Algorithm

- MFU page replacement algorithm replaces the page from memory which is most frequently used
- It is based on the argument that the page with the smallest count was probably just brought in and has yet to be used
- Its implementation is expensive
- Neither MFU nor LFU replacement algorithm is common

MFU – Example

Given

- Number of Page Frames = 3
- Reference String: 1 2 3 2 4 1 5 2 1 2 3 4

Reference: 1 2 3 2 4 1 5 2 1 2 3 4

Frame 1: 1 1 1 1 4 4 5 5 5 5 3 3

Frame 2: - 2 2 2 2 1 1 2 2 2 2 4

Frame 3: - - 3 3 3 3 3 3 1 1 1 1

Page Faults: ✓ ✓ ✓ ✗ ✓ ✓ ✓ ✓ ✓ ✗ ✓ ✓

Total Page Faults (MFU)

10 Page Faults